Appello 2 Settembre 2016



Politecnico di Milano
Anno accademico 2015-2016

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Ghezzi	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 5:

Esercizio 1

Una `TabellaSemiordinata` è un array associativo che mappa indici interi positivi a valori di un tipo `Dato`. Per il tipo `Dato` esiste una relazione di ordinamento definita da un opportuno `ComparatoreDato` che implementa l'interfaccia standard `Comparator`. Ad esempio, se `d1` e `d2` sono istanze di `Dato`, e `comparatore` è un'istanza di `ComparatoreDato`, allora `comparatore.compare(d1,d2)` ritorna un intero negativo se `d1` precede `d2`, un intero positivo se `d2` precede `d1`, o zero se `d1` e `d2` sono uguali.

Una istanza di `TabellaSemiordinata` deve garantire che quando viene aggiunto un nuovo elemento `dNew` in un indice `i` già occupato da un elemento `dOld`, l'elemento pre-esistente viene ri-assegnato al primo indice libero inferiore a quello originale se `dOld` precede `dNew`, oppure al primo indice libero superiore a quello originale se `dNew` precede `dOld`. Se invece il nuovo elemento `dNew` è uguale a `dOld`, la tabella non subisce variazioni.

L'interfaccia di `TabellaSemiordinata` è riportata di seguito.

```
public interface TabellaSemiordinata {

    boolean indiceOccupato(int d);

    void aggiungi(Dato d);

    void rimuovi(Dato d);

    Dato ritorna(int d);
}
```

Vi viene chiesto di:

1. Fornire la specifica JML di tutti i metodi definiti in `TabellaSemiordinata`.
2. Supponendo che la classe `MiaTabella` implementi `TabellaSemiordinata` utilizzando un oggetto `map` di tipo `Hasmap`, specificare la funzione di astrazione come private invariant.

SOLUZIONE

1. Specifica JML dei metodi: totalleftmargin=-2cm

```
public interface TabellaSemiordinata {

    //@requires d>=0
    //@ensures \result <==> ritorna(d)!=null
    boolean /*@ pure @*/ indiceOccupato(int d);

    /*@ requires d!=null && i>=0
    @ ensures indiceOccupato(i) && ritorna(i).equals(d) &&
    @ ((!\old(indiceOccupato(i)) ==>
    @   (\forall int idx; idx >= 0 && idx != i; indiceOccupato(idx) <==> \old(indiceOccupato(idx)) &&
    @                              indiceOccupato(idx)  ==> ritorna(idx).equals(\old(ritorna(idx))))) &&
    @  (\old(indiceOccupato(i))  ==>
    @    (comparatore.compare(d,\old(ritorna(i))) < 0 ==> (indiceOccupato(L) &&
    @                                                      ritorna(L).equals(\old(ritorna(i))) &&
    @       (\forall int idx; idx >= 0 && idx != i && idx != L; indiceOccupato(idx) <==> \old(indiceOccupato(idx))
    @                                 indiceOccupato(idx)  ==> ritorna(idx).equals(\old(ritorna(idx)))))) &&
    @    (comparatore.compare(d,\old(ritorna(i))) > 0 ==> (indiceOccupato(R) &&
    @                                                      ritorna(R).equals(\old(ritorna(i))) &&
    @       (\forall int idx; idx >= 0 && idx != i && idx != R; indiceOccupato(idx) <==> \old(indiceOccupato(idx))
    @                                 indiceOccupato(idx)  ==> ritorna(idx).equals(\old(ritorna(idx)))))) &&
    @    (comparatore.compare(d,\old(ritorna(i))) == 0 ==>
    @       (\forall int idx; idx >= 0 ; indiceOccupato(idx) <==> \old(indiceOccupato(idx)) &&
    @                                 indiceOccupato(idx)  ==> ritorna(idx).equals(\old(ritorna(idx))))))
    @ where L is \max(int idx; idx < i ; !\old(indiceOccupato(idx)))
    @ and   R is \min(int idx; idx > i ; !\old(indiceOccupato(idx)))
    @*/
    void aggiungi(Dato d);
```

```
    //@ requires d!=null
    //@ ensures (\forall int i; i>=0 && \old(ritorna(i)).equals(d); !indiceOccupato(i)) &&
    //@  (\forall int i; i>=0 && !\old(ritorna(i)).equals(d); indiceOccupato(i) <==> \old(indiceOccupato(i)) &&
    //@                              indiceOccupato(i)  ==> ritorna(i).equals(\old(ritorna(i))))) &&
    void rimuovi(Dato d);

    //@ requires d >= 0
    //@ ensures \result==null <==> !indiceOccupato(d)
    Dato /*@ pure @*/ ritorna(int d);
}
```

2. Funzione di astrazione (come private invariant) della classe `MiaTabella`:

```
public class MiaTabella implements TabellaSemiordinata {

    //@ private invariant map != null && !map.values().contains(null) &&
    //@  (\forall int i; map.keySet().contains(i); i>=0) &&
    //@  (\forall int i;; map.keySet().contains(i) <==> indiceOccupato(i) &&
    //@      (map.keySet().contains(i) ==> map.get(i).equals(ritorna(i))) &&
    private Map<Integer,Dato> map = new HashMap<Integer,Dato>();

    //...
}
```

Esercizio 2

Il sistema informativo di un'università deve gestire le informazioni relative ai propri dipendenti, dipartimenti, istituti, corsi erogati, e progetti di ricerca.

Le informazioni relative ai dipendenti comprendono la normale anagrafica (nome, cognome, data di nascita, ...). I dipendenti si dividono in ricercatori, attivi in un dato campo di ricerca, e personale amministrativo. Un dipendente può essere chiamato a dirigire un determinato dipartimento, che a sua volta comprende uno o più istituti.

I ricercatori fanno parte di uno o più istituti, e possono lavorare per un dato numero di ore al mese in determinati progetti di ricerca. Alcuni ricercatori sono anche impiegati come docenti di corsi erogati dall'università.

I corsi sono caratterizzati dal nome, il numero di ore totali, ed eventuali precedenze (obbligatorie o facoltative) rispetto ad altri corsi.
Vi viene richiesto di:

1. Creare un diagramma UML a vostra scelta per descrivere il sistema informativo.
2. Modificare il diagramma al punto precedente qualore il sistema informativo dovesse gestire anche le informazioni relative agli studenti e ai corsi già superati da uno studente.

Esercizio 3

Si consideri il seguente frammento di codice Java:

```
1.   public static int mistero(int i, int x) {
2.        int k = 1;
3.        while (k<x && x<=3)
4.             if (i > k || x == 1)
5.                  break;
6.             i = k/(x-1);
7.             k++;
8.        }
9.        return i;
      }
```

1. Si definisca il diagramma del flusso di controllo.

2. Si definisca l'insieme *minimo* di casi di test, supponendo che sia *i* che *x* siano non negativi, per coprire: (a) le istruzioni, (b) le decisioni (branch) e (c) le condizioni.

Qualora non si riesca ad arrivare ad una copertura completa, calcolare la percentuale di copertura.

SOLUZIONE

a) E' evidente che per entrare nel ciclo while, il valore di x deve essere 2 oppure 3. Per potere eseguire il break (istr. 5), occorre rendere vera la condizione dell'if: lo si puo' fare alla prima iterazione del ciclo, scegliendo un valore per la variabile i maggiore di k, ossia x¿1. Basta quindi scegliere ad esempio il caso (2,2) (ma anche (2,3) andrebbe bene).

Per potere eseguire le istruzioni 6 e 7, e' necessario che l'if sia falso, ossia che valga la condizione `i<=k && x!=1`: per entrare alla prima iterazione del ciclo (in cui k=1), basta quindi scegliere ad esempio i=2, ossia il caso (0,2) (ma anche (0,3), (1,2) e (1,3) andrebbero bene).

Questa scelta e' minimale, perche' non vi e' modo di rendere vero l'if dopo avere eseguito le istruzioni 6 e 7 (in quanto i diventa per forza minore di k), mentre dopo avere eseguito il break le istruzioni 6 e 7 saranno saltate. Quindi servono almeno due casi di test.

b) I due casi precedenti coprono anche il branch else "nascosto" dell'if. Sono quindi coperte anche tutte le diramazioni.

c) Con i due casi precedenti sono coperte anche le seguenti possibilita' per le costituenti delle condizioni:

```
k<x: TRUE, FALSE
x<=3:TRUE
i>k:FALSE, TRUE
x==1:FALSE
```

Per completare la copertura delle condizioni nell'AND nel while, serve coprire anche il caso `x<=3:FALSE` (naturalmente in questo caso `k<x:TRUE`), per il quale basta ad es. selezionare (0,4).

Per completare la copertura delle condizioni nell'OR dell'If, servirebbe coprire il caso il caso `x==1:TRUE`. Questo e' pero' evidentemente impossibile (se x, che non e' modificata, fosse uguale a 1 non si entrerebbe nel while, in quanto `k<=x` non sarebbe verificata). Quindi delle 8 possibilita' per le 4 costituenti delle condizioni, se ne coprono 7, ossia 87.5%.

Esercizio 4

Il seguente codice Java implementa una semplice versione della tradizionale interazione tra un thread produttore di dati ed un thread consumatore degli stessi dati, tramite una coda condivisa. La gestione delle eccezioni è omessa per brevità.

```
public class ProducerConsumer{
    public static void main(String args[]) {
        List<Integer> sharedQueue = new ArrayList<Integer>();
        Thread prodThread = new Thread(new Producer(sharedQueue), "Producer");
        Thread consThread = new Thread(new Consumer(sharedQueue), "Consumer");
        prodThread.start();
        consThread.start();
}}

class Producer implements Runnable {
private final List<Integer> sharedQueue;
public Producer(List<Integer> sharedQueue) {
  this.sharedQueue = sharedQueue;
}
public void run() {
  for (int i = 0; i < 10; i++) {
    System.out.println("Produced: " + i);
```

```
    produce(i);
  }
}
private synchronized void produce(int i) {
  sharedQueue.add(i);
  notifyAll();
}}

class Consumer implements Runnable {
private final List<Integer> sharedQueue;
public Consumer(List<Integer> sharedQueue) {
  this.sharedQueue = sharedQueue;
}
public void run() {
  while (true) {
    System.out.println("Consumed: " + consume());
  }
}
private synchronized int consume() {
  while (sharedQueue.isEmpty())
    wait();
  notifyAll();
  return sharedQueue.remove(0);
}}
```

Vi viene chiesto di:

1. Argomentare con precisione il motivo per cui questa implementazione risulta corretta rispetto alla sincronizzazione di produttore e consumatore. Viceversa, se l'implementazione presenta qualche problema, descrivere con precisione il problema e a quali conseguenze può portare. Descrivere inoltre (mediante codice Java) le modifiche al codice necessarie per rimediare al problema. **SOLUZIONE**

   I due thread non sono fra loro sincronizzati! Una soluzione e' la seguente: Nel Consumer:

```
....
    private int consume() throws InterruptedException {
        while (sharedQueue.isEmpty()) {
            synchronized (sharedQueue) {
                sharedQueue.wait();
            }
        }

        synchronized (sharedQueue) {
            sharedQueue.notifyAll();
            return sharedQueue.remove(0);
        }
    }
}
```

   e nel Producer:

```
...
    private void produce(int i) throws InterruptedException {

        synchronized (sharedQueue) {
```

```
                sharedQueue.add(i);
                sharedQueue.notifyAll();
            }
        }
```

2. Independentemente dalla risposta alla domanda precedente, modificare l'implementazione in maniera che la coda condivisa non possa crescere arbitrariamente, ma possa al massimo contenere `MAX` elementi, dove `MAX` è una costante intera. **SOLUZIONE**
   Si puo' modificare il producer in questo modo, per metterlo in attesa quando il numero di elementi prodotti supera il massimo: totalleftmargin=-2cm

```
private void produce(int i) throws InterruptedException {

        //wait if queue is full
        while (sharedQueue.size() == size) {
            synchronized (sharedQueue) {
                System.out.println("Queue is full " + Thread.currentThread().getName()
                                    + " is waiting , size: " + sharedQueue.size());

                sharedQueue.wait();
            }
        }

        //producing element and notify consumers
        synchronized (sharedQueue) {
            sharedQueue.add(i);
            sharedQueue.notifyAll();
        }
    }
}
```

   e corrispondentemente si modifica il consumer come segue: totalleftmargin=-2cm

```
...
private int consume() throws InterruptedException {
     //wait if queue is empty
     while (sharedQueue.isEmpty()) {
      synchronized (sharedQueue) {
          System.out.println("Queue is empty " + Thread.currentThread().getName()
               + " is waiting , size: " + sharedQueue.size());

          sharedQueue.wait();
         }
     }

     //Otherwise consume element and notify waiting producer
     synchronized (sharedQueue) {
         sharedQueue.notifyAll();
         return sharedQueue.remove(0);
     }
 }
```

Nel rispondere alle domande, non ci si preoccupi della gestione delle eventuali eccezioni.

Esercizio 5

Trasformare il seguente codice Java in stile funzionale.

```
public static void numeroPari(final List<Integer> numeri) {

  Integer found = null;
  for (Integer i: numeri) {
    if (i % 2 ==0) {
      found = i;
      break;
```

```
    }
  }
  if (found != null) {
    System.out.print("Il primo numero pari " + found);
  } else {
    System.out.println("Nessun numero pari!");
  }
}
```

SOLUZIONE

Una soluzione semplice e':

```
package Functional;

import java.util.List;
import java.util.Optional;

public class Functions {
public static void numeroPari(final List<Integer> numeri) {
final Optional<Integer> trovato = numeri.stream().filter(i -> (i % 2 == 0)).findFirst

if (trovato.isPresent())
System.out.println("Il primo numero pari" + trovato);
else
System.out.println("Nessun numero pari!");
}
}
```